

Class Relationships

Types of Relationships

Relationships are classified as follows:

- A Kind-Of relationship
- An Is-A relationship
- A Part-Of relationship
- A Has-A relationship

Super Class and Sub Class

Example: Let us take the example of rail tickets.

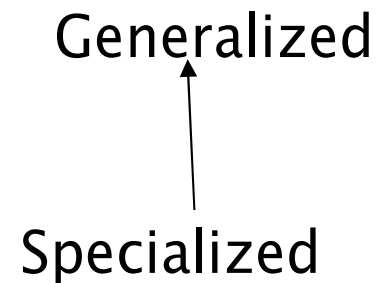
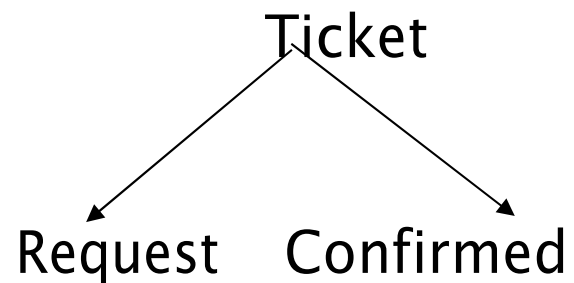
Rail tickets can be of two types.

- Confirmed and Request.
- Both the tickets have a lot of common attributes, for example, Train number, data, time and destination.
- However, a confirmed ticket would also have a seat number, while a Request ticket would have a status.

SYNTAX

```
class subclass-name extends superclass-name  
{  
  // body of class  
}
```

Example: Real Time



```
class A
{
int i=4,j=5;
void show1()
{
System.out.println("I and j:"+i+j);
}
}

class B extends A
{
int k=10;
void show2()
{
System.out.println("k:"+k);
}
```

```
void sum()
{
    System.out.println("I  +j+k:"+(i+j+k));
}
}
```

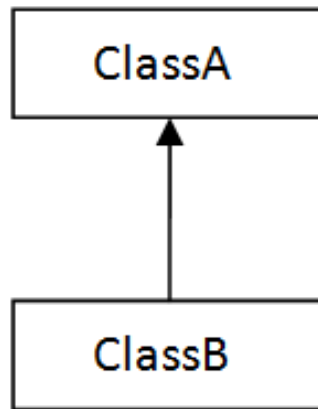
```
class SimpleInheritance
{
    public static void main(String args[])
    {
        B superob=new B();
        superob.show1();
        superob.show2();
        superob.sum();
    }
}
```

```
class A
{
int i; // public by default
private int j; //private to A
void set(int x, int y)
{
i=x;
j=y;
}
}
```

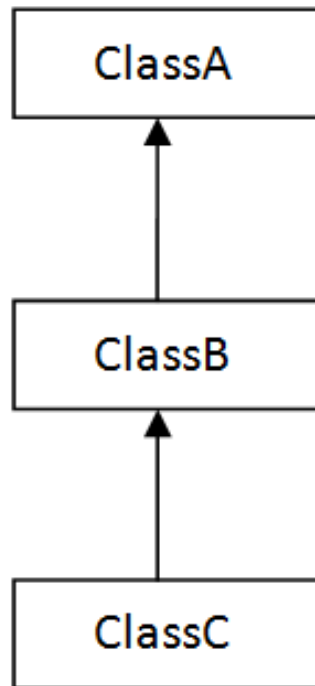
```
class B extends A
{
int total;
void sum()
{
total=i+j; // Error, j is not accessible here
}
}
```

Types of Inheritance

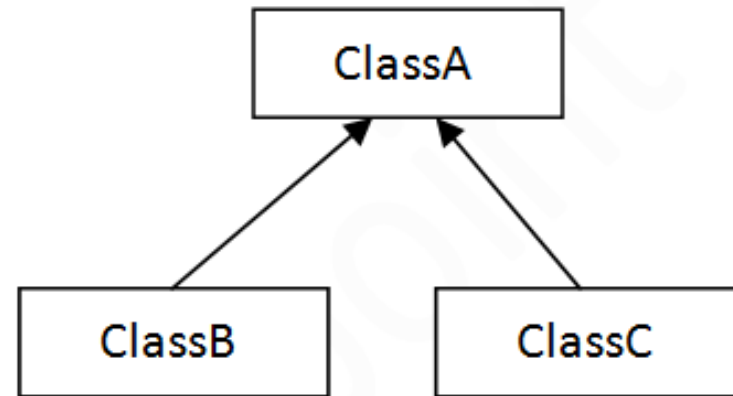
1. Single inheritance
2. Multilevel inheritance
3. Hierarchy inheritance
4. Multiple inheritance—Java does not support directly by class
5. Hybrid inheritance—Java does not support



1) Single



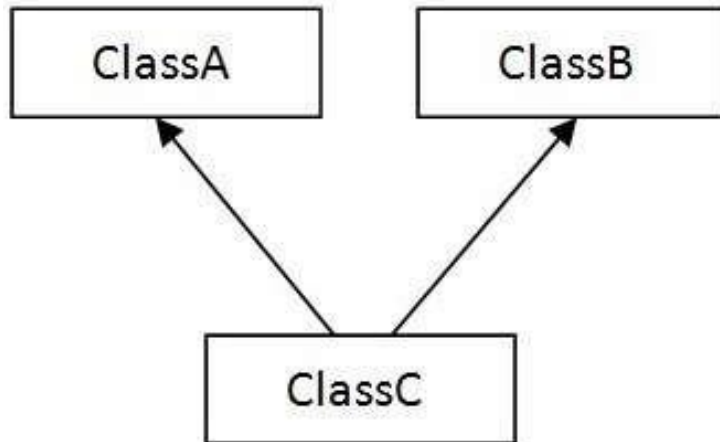
2) Multilevel



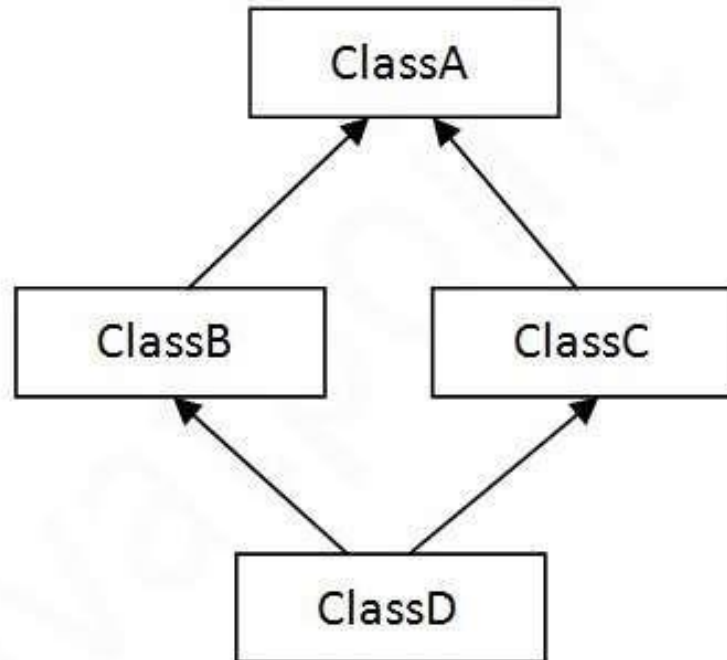
3) Hierarchical

Types of Inheritance

1. Single inheritance
2. Multilevel inheritance
3. Hierarchy inheritance
4. Multiple inheritance—Java does not support directly by class
5. Hybrid inheritance—Java does not support



4) Multiple



5) Hybrid

Example of Inheritance

```
class A{  
void msg(){System.out.println("Hello");}  
}  
class B{  
void msg(){System.out.println("Welcome");}  
}  
class C extends B{  
  
    Public Static void main(String args[]){  
        C obj=new C();  
        obj.msg();//Now which msg() method would be invoked?  
  
    }  
}
```

Types of Polymorphism

1. Compile time polymorphism or static binding or early binding

Ex: Overloading

2. Runtime polymorphism or Dynamic binding or Late binding

Ex: Overriding

Types of Overloading

Method overloading

Constructor Overloading

//Method overloading

```
class TestAdd{  
    void sum(int a,int b){System.out.println(a+b);}  
    void sum(int a,int b,int c){System.out.println(a+b+c);}  
    public static void main(String args[]){  
        TestAdd obj=new TestAdd();  
        obj.sum(10,10,10);  
        obj.sum(20,20); } }
```

Constructor Overloading

```
class sum{  
    void sum(int a,int b){System.out.println(a+b);}  
    void sum(int a,int b,int c){System.out.println(a+b+c);}  
    public static void main(String args[]){  
        new sum(1,2);  
        new sum(1,2,3);  
    } }
```

2.Run time polymorphism

Overriding

```
class A{  
void msg(){System.out.println("Hello");}  
}  
class B{  
void msg(){System.out.println("Welcome");}  
}  
class C extends B{  
  
Public Static void main(String args[]){  
    C obj=new C();  
    obj.msg();//Now which msg() method would be invoked?  
}  
}
```

Dynamic Method Dispatch to avoid Overriding

```
class A{  
void msg(){System.out.println("Hello");}  
}  
class B{  
void msg(){System.out.println("Welcome");}  
}  
class C extends B{  
  
Public Static void main(String args[]){  
    A obj=new A();  
    obj.msg();  
    obj=new B();  
    obj.msg();  
  
}  
}
```

Types of class

1. Local or inner class
2. Anonymous class

Example of Local Class

```
class Te{// this is public
    void disp(){
        System.out.println("am main class");
        Local o=new Local();
        o.add();
    }
    class Local{// local class or inner class
        void dsp(){
            System.out.println("Hello world");
        }
    }
}
```

```
classs MainClass{  
  
    public static void main(String ...as){  
  
        Te obj=new Te();  
        Te.Local in=obj.new Local();  
        in.dsp();  
    }  
}
```

2. Anonymous class --without object we can access the method.

```
abstract class Test{  
    abstract void disp();  
}
```

Test x=new Test();// can't create object for interface and abstract class.

```
class Test1{  
    public static void main(String ...d){  
        Test anony=new Test(){  
void disp(){  
        System.out.println("HELLO ");  
        } };  
        anony.disp();  
    }  
}
```

Access Modifier & Other Modifier

- **Access Modifier:** we can applying access modifier the following:
- VARIABLE ,CLASS,METHODS AND OBJECT.

- 1.**public:** We can apply to variable–class–method–object :Any where we can use.
- 2.**protected:** We can apply to class–method–variable :We can use with in package
- 3.**private:** We can apply to variable –method– object, class : only in class.
4. **default:** We can apply to no keyword as a default. :With in package.

- **Other Modifier:** we can applying access modifier the following:

final–we can use class, variable, method ,object.

static–we can use class, variable ,method ,object.

abstract– we can use class, variable, method.

- **final Modifier:**

1. final class can't inherited.

2. final variable should be constant.

Ex:

```
final class T{  
final int i=10;  
}
```

```
class A extends T{// Error because "T" class is final.  
}
```

```
class A{  
public static void main(String...as){  
T obj=new T();  
obj.i=20;// Error because "i" is final.  
}  
}
```

- **static Modifier:**

static should be create **unique object**.

```
class T1{  
    static int i=10;
```

```
    void disp(){  
        System.out.println("am a method");  
    }
```

```
    static{ System.out.println("am block");  
    }
```

```
    public static void main(String...as){  
        System.out.println(i); // without object we can accessing...  
    }  
}
```

- **abstract Modifier:**

- Abstract method **must be override** to your sub class.

```
abstract class T1{  
    abstract void disp();  
}
```

```
Class A extends T1{  
    public void disp(){  
        System.out.println("This method overriding");  
    }  
}
```

```
public static void main(String...as){  
    A ob=new A();  
    ob.disp();  
}  
}
```

Super and This Keyword

- “super” -This is refer the super class object.
- “this” -This is refer the current class object.

```
class T1{  
    void disp(){  
        System.out.println("am from T1");  
    }  
}
```

```
class T2 extends T1{  
    void disp(){  
        super.disp();// o/p: am from T1—super call the super class object  
        System.out.println("am from T2");  
    }  
}
```

```
Class A extends T2{  
    public static void main(String...as){  
        A ob=new A();  
        ob.disp();// o/p: am from T2  
    }  
}
```

- “this” - This is refer the current class object.

```
class T1{  
    Int i=10;// this “I” in class...  
    void disp(int i){// this “I” is in method...  
        this.i=i;//  
        System.out.println(i);  
    }  
}
```

```
Class A extends T2{  
    public static void main(String...as){  
        A ob=new A();  
        ob.disp(20);// o/p: is 20  
    }  
}
```

- Test the all inheritance concepts.
- Test the overriding concepts.
- Test the dynamic method dispatch mechanism in java.